UNIDAD 1 - SEMANA 2
RESUMEN DE CLASE

Nombre: Doménica Rojas
Ciclo: Quinto “A”
Asignatura: Sistemas Digitales

Tema: Del AVR al ESP32

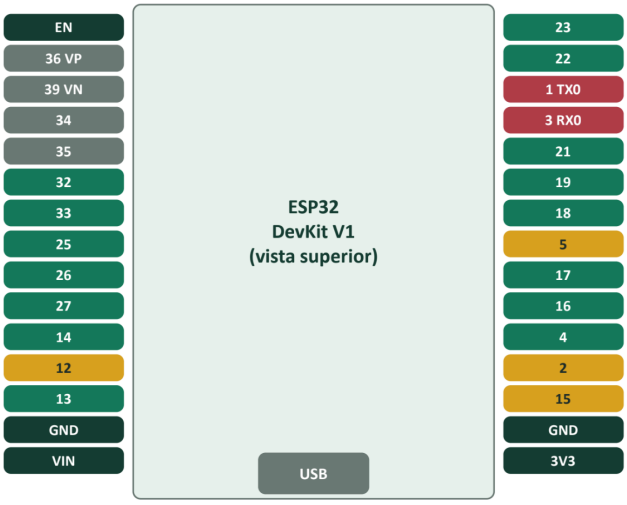
1. Registros GPIO y uso correcto de los pines

El ESP32 mantiene la misma idea del AVR: controlar una entrada o salida implica leer o escribir bits en registros. La diferencia es que tiene 32 bits, una matriz GPIO configurable y registros con otros nombres.

Función	AVR	ESP32
Configurar dirección	DDRB	GPIO_ENABLE_REG
Poner salida en alto	PORTB	GPIO_OUT_W1TS_REG
Poner salida en bajo	PORTB	GPIO_OUT_W1TC_REG
Leer entrada	PINB	GPIO_IN_REG

Los registros W1TS (write 1 to set) y W1TC (write 1 to clear) modifican solamente los bits marcados con 1 mediante una escritura atómica. Así se evitan problemas de leer-modificar-escribir cuando una interrupción u otro núcleo cambia el registro.

- **GPIO recomendados para LED o pulsador:** 16, 17, 18, 19, 21, 22, 23, entre otros mostrados en el mapa de pines.
- **GPIO 34–39:** solo entrada; no cuentan con pull-up interna.
- **GPIO 0, 2, 5, 12 y 15:** pines de arranque (strapping) conviene evitar conectarlos de forma que alteren el arranque.
- **GPIO 1 y 3:** usados por UART0.
- **GPIO 6–11:** reservados para la memoria Flash.
- **Tensión lógica:** 3,3 V. No hay que conectar directamente señales de 5 V a las entradas.



2. Cómo se ejecuta un programa embebido

En Arduino, `setup()` se ejecuta una vez para inicializar pines y servicios, mientras `loop()` repite continuamente la lectura de entradas, las decisiones y la escritura de salidas (superlazo). En el ESP32 esta estructura funciona sobre FreeRTOS.

Una variable local común se reinicia al entrar de nuevo a la función; una `static` mantiene su valor entre llamadas; una global también conserva su estado mientras el programa funciona. Un ciclo vacío consume CPU, por lo que se debe considerar la eficiencia energética.

3. Tipos de datos y operaciones seguras

El tamaño de int cambia entre plataformas: en el Arduino Uno ocupa 2 bytes y en el ESP32 4 bytes. Por eso es preferible declarar tipos de ancho fijo: `uint8_t` (0–255), `uint16_t` (0–65 535) y `uint32_t` (0–4 294 967 295), especialmente para registros, contadores y tiempo.

- Desbordamiento sin signo: un uint8_t pasa de 255 a 0; un entero con signo que desborda no ofrece el mismo comportamiento definido.
- Promoción de tipos: la operación ~a puede convertirse a int antes de invertirse; usar (uint8_t)~a cuando se necesita un resultado de 8 bits.
- Máscaras de 32 bits: usar 1UL << n para evitar problemas al desplazar el bit alto.
- `constexpr`: constante cuyo valor se conoce al compilar. `volatile`: avisa al compilador de cambios externos, pero no hace atómicas las operaciones.

4. Temporización no bloqueante con millis()

delay() detiene el avance del loop(), por lo que una pulsación rápida puede pasar inadvertida. `millis()` permite comprobar cuánto tiempo transcurrió y seguir atendiendo otras tareas. La comparación por resta también funciona cuando el contador sin signo da la vuelta:

```
uint32_t ahora = millis();
if (ahora - tAnterior >= PERIODO) {
   tAnterior += PERIODO;
   // ejecutar la accion periodica
}
```

No se recomienda comparar millis() > tAnterior + PERIODO, porque la suma puede desbordarse y adelantar la activación.

5. Pulsadores: pull-up, rebote y flancos

Con **`INPUT_PULLUP`**, el pulsador conectado a GND lee HIGH al estar suelto y LOW cuando se presiona. El contacto mecánico genera **rebotes**, por lo que se acepta un cambio solo cuando permanece estable unos **20–50 ms** (en el ejemplo de la clase, 30 ms).

Conviene detectar el **flanco de bajada** (transición de HIGH a LOW) y contar una sola vez cada pulsación, en lugar de incrementar mientras el botón permanezca en LOW. La práctica combina lectura de entradas, estado, tiempo con millis() y actualización de LEDs.